# AgroSmart – Análisis de Algoritmos y Estructuras de Datos

## Módulo: Calendario y predicción de cosecha (calendario.py)

## 1. Estructuras que usa el módulo

| Estructura | Dónde | Para qué |
|---|---|---|
| Lista | `self.eventos` | Guarda todos los eventos registrados. |
| **Árbol binario de búsqueda** | `ArbolEventos` (`estructuras/arbol_eventos.py`) | Mantiene los eventos ordenados por fecha. |
| Diccionario (tabla hash) | `self.dias_maduracion` | Busca los días de maduración de cada tipo de cultivo. |
| Archivo JSON | `eventos_calendario.json` | Guarda los eventos para que no se pierdan. |

El módulo **no usa ninguna conexión a internet**: la predicción se calcula con la fecha de siembra del cultivo y una tabla de días de maduración.

## 2. Eventos ordenados con un árbol binario de búsqueda

**Explicación.** Cada vez que se agrega un evento, se guarda en la lista y también se inserta en el árbol. En el árbol, los eventos con fecha menor van hacia la izquierda y los demás hacia la derecha. Para mostrar la agenda en orden se hace un **recorrido en orden** (izquierda, nodo, derecha), que entrega los eventos ordenados por fecha sin tener que ordenar la lista cada vez.

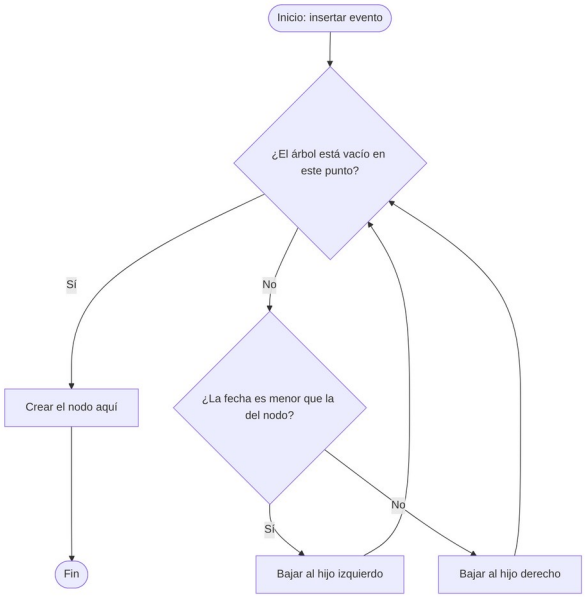
**Pseudocódigo**

```
FUNCION insertar(raiz, evento)
    SI raiz es NADA: DEVOLVER nodo(evento)
    SI evento.fecha < raiz.evento.fecha
        raiz.izquierdo = insertar(raiz.izquierdo, evento)
    SINO
        raiz.derecho = insertar(raiz.derecho, evento)
    FIN SI
    DEVOLVER raiz
FIN FUNCION

FUNCION en_orden(nodo, resultado)
    SI nodo es NADA: TERMINAR
    en_orden(nodo.izquierdo, resultado)
    AGREGAR nodo.evento A resultado
```

```
    en_orden(nodo.derecho, resultado)
FIN FUNCION
```

**Diagrama de flujo (insertar)**




**Complejidad.** Insertar: O(log n) en promedio, O(n) si los eventos llegan ya ordenados y el árbol queda inclinado. Recorrido en orden: O(n). Espacio: O(n).

**Justificación.** El calendario siempre se muestra ordenado por fecha. Con el árbol, cada evento nuevo queda en su lugar al insertarlo, y no hay que reordenar toda la lista cada vez que se agrega una actividad.

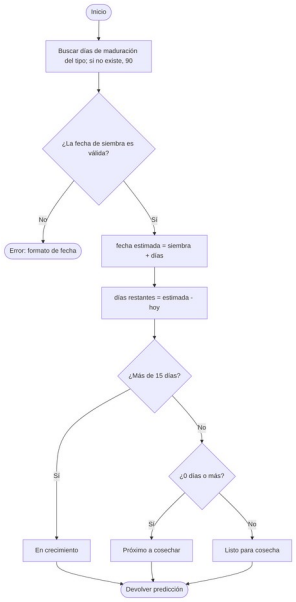
## 3. Predicción de cosecha

**Explicación.** `calcular_prediccion` busca en el diccionario los días que tarda el cultivo en madurar (por ejemplo, maíz 120, frijol 75, tomate 90, café 270, papa 110, arroz 120, hortalizas 60). Si el tipo no está en la tabla, usa 90 días. Suma esos días a la fecha de siembra para obtener la fecha estimada, calcula los días restantes respecto a hoy y asigna un estado.

**Pseudocódigo**

```
FUNCION calcular_prediccion(codigo, tipo, fecha_siembra)
    dias_ciclo = dias_maduracion[tipo en minúsculas] o 90 si no existe
    SI fecha_siembra no tiene formato AAAA-MM-DD: ERROR
    fecha_estimada = fecha_siembra + dias_ciclo
    dias_restantes = fecha_estimada - hoy
    SI dias_restantes > 15:        estado = "En crecimiento"
    SINO SI dias_restantes >= 0:   estado = "Próximo a cosechar"
    SINO:                          estado = "Listo para cosecha"
    DEVOLVER (fecha_estimada, MAXIMO(0, dias_restantes), estado)
FIN FUNCION
```

**Diagrama de flujo**




**Complejidad.** Tiempo O(1): una búsqueda en el diccionario, una suma de fechas y comparaciones fijas. Espacio O(1).

**Justificación.** Se eligió un modelo simple por días de ciclo porque es fácil de explicar, no depende de internet y se puede ajustar cambiando la tabla. La búsqueda en diccionario es O(1) en promedio porque Python lo implementa como tabla hash.

## 4. Validación y normalización de fechas

**Explicación.** Antes de guardar, la fecha se pasa por `normalizar_fecha` (`fechas.py`), que valida el formato y la deja siempre con ceros (2026-10-5 pasa a 2026-10-05). Esto es importante porque las fechas se comparan como texto en el árbol y al ordenar: con ceros, el orden alfabético coincide con el orden cronológico. Al cargar el archivo se usa `normalizar_fecha_segura` para corregir fechas viejas sin detener el programa.

**Complejidad.** O(1).

## 5. Código de cada evento y filtro por cultivo

- **Código nuevo (EVT001, EVT002...):** se recorre la lista de eventos para encontrar el número más alto y se suma 1. Tiempo O(n).
- **Filtro por cultivo:** se recorre la lista que entrega el árbol (ya ordenada) y se quedan los eventos del cultivo seleccionado. Tiempo O(n). Como el árbol ya entrega los eventos ordenados, el filtro conserva ese orden.

## 6. Guardado y carga (JSON)

Cada evento nuevo reescribe `eventos_calendario.json`. Al iniciar, `cargar_datos` lee el archivo, vuelve a insertar cada evento en la lista y en el árbol, y si el archivo está dañado muestra un aviso en lugar de cerrar el programa.